

The Pokémon Champions RAG playbook: a prescriptive 2024–2026 guide

This playbook is a reference manual for a Claude agent working on the Pokémon Champions Expert RAG system (Supabase pgvector + TSVECTOR + RRF, Xenova MiniLM-L6, TypeScript/Node on Vercel, Claude Code slash commands). It is opinionated: every section ends with a call on what to do for *this* project, not a generic survey.

Bottom line up front. The system is architecturally sound — hybrid RRF, structured-SQL fallback for stat queries, an agentic tool loop with dedup and forced completion — but it has three acute weaknesses: (1) the eval suite is **saturated at MRR 1.0 on 25 easy cases** and blind to generation-side faithfulness, (2) the **embedding model (MiniLM-L6-v2) is ~15 nDCG points behind modern 384-dim-compatible SOTA**, and (3) the retrieval stack has **no reranker**, which is the single highest-ROI upgrade available. The highest-leverage next three actions are: **add Contextual Retrieval at ingestion time** (Anthropic, -35–49% failure), **add a cross-encoder or Jina ColBERT-v2 reranker over top-40 → top-8**, and **replace the 25-case saturated eval with a stratified ~200-case suite (Giskard RAGET + hard-negative mining) plus faithfulness scoring via a Python RAGAS offline job piped into Langfuse**. Everything else in this document is downstream of those three.

1. What the project already has right

Before prescribing changes, name the existing patterns that are already best-practice as of 2026 so the agent does not regress them:

- **Hybrid dense + sparse with RRF (k=60).** Cormack-Clarke-Buettcher 2009 RRF at k=60 is the de-facto default and empirically robust; [Spice AI](#) tuning k is rarely worthwhile until you have calibrated per-query behavior. The existing single-RPC architecture is the right place for it.
- **Parallel structured-SQL path for stat-threshold queries.** This is the correct design — exact predicates (`BST > 500`), (`Speed >= 130`) should never be expressed as vector similarity. Self-querying retrievers are the LLM-based generalization of the same idea; [DEV Community](#) rules beat LLMs here because they are 1 ms, free, and deterministic.
- **Rule-based intent classifier with word-boundary matching against Pokémon/move dictionaries.** Dominant practice for well-enumerated intents. Keep for the high-frequency path; only add LLM-based classification as a long-tail fallback.
- **8-boost additive re-ranker calibrated to RRF scale (~0.02–0.035).** This is a pragmatic metadata-aware re-ranker and it is fine for now. It is *not* a cross-encoder and cannot replace

one; think of it as a cheap metadata bias layer on top of whatever semantic reranker you add.

- **Agentic tool loop with dedup, pokedex guardrails, post-loop force-completion.** These are exactly the failure modes identified in the 2024–2026 agentic RAG literature (retrieval loops, citation hallucination, over-retrieval). Keep them all.
- **Staleness detection via `file_mtimes` in `pc_index_meta`.** Good pattern, retain.
- **CSV → NL per-row conversion with `data_category` tags.** Correct — embedding raw CSV rows wastes semantic signal because the encoder has no schema awareness.

Things that are **outdated or suboptimal** (addressed below): MiniLM-L6-v2 as the embedder, `vector(384)` (should be `halfvec(384)`), chunk-time Italian translation via a 2,383-entry dictionary, eval suite saturation, no cross-encoder reranker, no contextual chunk enrichment, no groundedness / citation-validation loop, no prompt caching on the Claude fallback path.

2. Evaluation methodology: escaping MRR 1.0

Why MRR 1.0 on 25 cases means nothing

MRR rewards the retriever for surfacing a known positive at rank 1 on queries that are already lexically matchable — which is precisely the regime where hybrid RRF with named-entity BM25 always wins. The metric is blind to: generation faithfulness, context precision beyond rank 1, adversarial distractors, unseen intents, per-category failure modes, and drift. Before anything else, **retire MRR-only as the primary signal.**

The minimum viable replacement suite

Adopt four metric families simultaneously, because each covers a different failure mode:

1. **Retrieval metrics with graded relevance.** Replace binary relevance with {0, 1, 2, 3} and compute **nDCG@10, Recall@10, Context Precision@10**. nDCG penalizes partial matches and does not saturate on small sets the way MRR does.
2. **Faithfulness / groundedness.** Decompose each generated answer into claims; check each claim against retrieved context. RAGAS computes this as $\frac{|\text{claims supported by context}|}{|\text{total claims}|}$. `Ragas Ragas` A healthy RAG should sit at 0.85+; anything below 0.7 is hallucinating.
3. **Answer relevancy.** RAGAS's reverse-question method: generate N hypothetical questions the answer addresses, score mean cosine similarity to the original query. `Articlsledge` Detects evasive or off-topic generation.

4. **Per-intent slice metrics.** Compute all of the above separately for each of the project's seven intents (`usage`, `counter`, `stat`, `item`, `move`, `team`, `matchup`). A global average hides the fact that `counter` queries may have 0.5 faithfulness while `stat` sits at 0.95.

Frameworks — what to actually adopt on a TypeScript stack

Almost every serious RAG eval framework (RAGAS, RAGChecker, ARES, DeepEval, Giskard RAGET, TruLens) is **Python-only**. The right architecture on a Node project is a **split stack**: Langfuse (or Arize Phoenix) handles tracing and online scoring natively in TypeScript, and a nightly Python job running RAGAS/DeepEval/RAGChecker consumes a JSONL export of traces and pushes scores back via the Langfuse API.

- **Langfuse TS SDK** (`@langfuse/tracing`, `@langfuse/otel`, `@langfuse/client`) — **top pick**. First-class TypeScript, OpenTelemetry-based, `Langfuse` `Langfuse` Vercel AI SDK integration, supports scores, datasets, experiments, prompt management, online LLM-as-judge evaluators. Critical Vercel caveat: always `await langfuse.shutdownAsync()` in the serverless handler `Langfuse` or events drop silently. `Langfuse`
- **Arize Phoenix** is the strong alternative with `@arizeai/phoenix-client`, `@arizeai/phoenix-otel`, `@arizeai/phoenix-evals` TS packages `Arize` and native Vercel AI SDK instrumentation. Pick it over Langfuse if the team already commits to OpenInference semantic conventions.
- **RAGAS** (arXiv 2309.15217) for faithfulness, answer relevancy, context precision/recall. `Ragas` Run as a Python GitHub Actions job reading a JSONL dump. v0.2+ uses a collections API; v0.1 is deprecated.
- **RAGChecker** (arXiv 2408.08067) for claim-level entailment `ResearchGate` diagnostics — gives you the cleanest view of whether errors are in retrieval (claim recall low) or generation (hallucination high). Requires ground-truth answers. `PyPI` Run monthly, not per-PR.
- **DeepEval** for pytest-style `GitHub` CI gates with metrics like `GEval`, `Medium` `FaithfulnessMetric`, `ContextualPrecisionMetric`, `Deepeval` and its **DAG metric** for deterministic checks (e.g., "every answer must cite ≥ 1 chunk_id" as a hard graph condition). `Deepeval`
- **Giskard RAGET** — **the single highest-ROI eval tool for escaping saturation**. It auto-generates a stratified testset of six question types (simple, complex/paraphrased, distracting, situational, double, conversational) `KNIME Community Hub` from the corpus `Giskard` and reports `correctness_by_question_type` and `correctness_by_topic`. Pin `giskard[llm]>=2,<3` because v3 dropped RAGET. `GitHub`

1. **Graded relevance.** Relabel the existing 25 cases with {0, 1, 2, 3}. nDCG@10 will drop below 1.0 immediately because partial matches now carry information.
2. **Hard-negative mining.** Following the two-condition criterion from Ritvik19/arXiv 2505.18366, select chunks D such that $d(Q, D) < d(Q, \text{positive})$ and $d(\text{positive}, D)$ falls in a bounded band — semantically close to the query but not answering it. For Pokémon: "Garchomp moveset" has as a hard negative "Garchomp ability description," as a harder negative "Flygon moveset" (same family, wrong entity), and as an adversarial negative "Garchomp lore / Pokédex flavor text." Inject these into the corpus and fail any run where the retriever mixes them in the top 5.
3. **Failure-driven expansion from production logs.** Mine Langfuse traces where user feedback is negative or where the online LLM-as-judge flags faithfulness < 0.7. Cluster, label, permanently add to the golden set. This creates a regression pool that grows with real failure modes rather than with developer imagination.
4. **Perturbation / adversarial testing.** For every golden query, auto-generate: typo variants, synonym swaps, entity-rename variants ("Landorus-T" → "Lando-T" → "Incarnate Landorus"), paraphrases, abbreviation expansion, negation insertion. A robust system drops $\leq 3-5$ nDCG points under perturbation.
5. **Difficulty tiers.** Stratify the suite into Easy (single-hop, exact match), Medium (paraphrase, synonym), Hard (multi-hop, 2+ chunks required), Adversarial (distractors present). Report per-tier. The current MRR 1.0 is on Easy only.
6. **Noise sensitivity (RAGChecker).** Inject irrelevant chunks into the context at eval time; score drop = noise sensitivity. A healthy RAG is roughly invariant.

Synthetic test generation

- **Giskard RAGET** is the fastest path to ~200 stratified cases. Run once, import into Langfuse datasets, iterate.
- **RAGAS** `TestsetGenerator` takes documents + generator LLM + critic LLM + embeddings and produces a mix of `simple / reasoning / multi_context / conditional` questions via an Evol-Instruct-style pipeline. `Ragas` `GitHub` Default distribution `{simple: 0.5, reasoning: 0.25, multi_context: 0.25}`; `Ragas` start at `test_size=100-200`.
- **DIY from corpus:** for each chunk, prompt "generate N questions this chunk uniquely answers," then filter with a verifier LLM "does this chunk answer this question?". Reject low-confidence. Add cross-chunk distractors.

- **Crucial leakage rule:** never use the same LLM for synthetic generation, system generation, and judging. Self-enhancement bias inflates scores.

LLM-as-judge best practices

- Prefer **pairwise preference** (A vs B) over pointwise scoring for A/B tests — more stable across model versions. Use pointwise only for dashboards with a stable pinned judge.
- Use **G-Eval** (arXiv 2303.16634) — auto-generated chain-of-thought evaluation steps plus form-filling (arXiv) with token-probability-weighted scores. Spearman $\rho=0.514$ with humans on SummEval. (arXiv)
- **Mitigate every known bias explicitly:**
 - *Position bias* (arXiv 2406.07791): always run pairwise in both orders (A,B) and (B,A), average. (Sebastiansigl) (OpenReview)
 - *Verbosity bias*: include "penalize unnecessary length" in the rubric (arXiv) or length-normalize.
 - *Self-enhancement*: use a judge from a different model family than the generator (arXiv) (e.g., Claude judges gemma-4-26b output; GPT-4 judges Claude output).
 - *Chain-of-thought bias*: a judge is influenced by the presence of reasoning traces — use DAG / deterministic nodes for objective criteria (citation presence, format) and reserve LLM judgment for subjective ones.
- Decompose global judgments into claim-level (RAGChecker, FactScore arXiv 2305.14251) whenever possible. "Rate this answer 1-5" is the weakest LLM-as-judge pattern.

CI / regression patterns

- **Golden set** as version-controlled JSONL: ((input, expected_output, expected_contexts, intent_tag, difficulty_tier)). Start ~100, grow to ~500.
- **PR gate**: fail if any tier-level metric drops > 5% from (main) or any critical test flips. DeepEval's (assert_test) + GitHub Actions is the idiomatic pattern; (Deepeval) (Confident AI) Langfuse Experiments via REST is the TS-native alternative.
- **Drift detection**: embed every production query, track centroid cosine distance vs. a reference window. Langfuse/Phoenix both compute aggregate score timeseries.
- **Shadow replay**: periodically re-run historical production queries through the current system; compare to stored baselines. Any drop > threshold triggers a human review.

Production / online evaluation

- Explicit user feedback: `langfuse.score({ traceId, name: "user_thumbs", value: 1 })` (Langfuse) from the chat UI via (LangfuseWeb). (Langfuse)
- Implicit signals: abandonment, retry rate, copy-button clicks, citation click-through — log as custom scores, correlate with LLM-as-judge to validate the judges themselves.
- A/B tests: route users to variants by flag; tag traces with (variant); compare faithfulness + answer_relevancy + user feedback with $n \geq$ several hundred per arm before declaring a win.

What to add to the existing 13-test agentic eval harness

The harness covers end-to-end correctness on 13 scenarios. The gaps, in priority order:

1. **Trajectory metrics.** (tool_call_count), (unique_chunk_ids_used), (retrieval_precision@k) per test. Catches loops and over-retrieval that the final-answer check can miss.
2. **Groundedness score.** Fraction of claims in the final answer that are traceable to a retrieved chunk — implement as a cheap Claude Haiku post-hoc check.
3. **(pass^3) consistency.** Run each test three times with (temperature > 0) and measure whether the system agrees with itself. Borrowed from τ -bench (Sierra, arXiv 2406.12045); the single most important agent-reliability metric in 2025.
4. **Cost/latency budgets per test.** Regress if a new prompt bloats tokens or wall time.
5. **Cache-hit regression.** Verify (usage.cache_read_input_tokens > 0) on expected-cached calls so prompt-caching layout doesn't silently degrade.
6. **Adversarial cases.** Prompt-injection-in-retrieved-chunk, phantom Pokémon with similar names, typos. The (phantom_pokemon) case is a good start; expand deliberately.

3. Retrieval improvements

Priority 1: Add Contextual Retrieval at ingestion

This is the highest-ROI retrieval change available for this project. **Anthropic's Contextual Retrieval** (anthropic.com/news/contextual-retrieval, Sep 2024) prepends a 50-100 token LLM-generated context to each chunk *before* embedding and BM25 indexing. (CO/AI) Reported top-20 retrieval-failure reductions averaged across codebases, fiction, arXiv, and science:

- Contextual Embeddings alone: **-35%**. (CO/AI +3)

- Contextual Embeddings + Contextual BM25: **-49%**. (CO/AI +3)
- Plus Cohere reranker on top: **-67%**. (Anthropic +4)

Why it matters for this project. Pokémon chunks are often semantically thin: "Deals 40 base power. Hits twice." is almost useless standalone. Prepending *"This chunk describes the move Double Kick, a physical Fighting-type move learned by Nidoran-M and 40+ other Pokémon."* dramatically improves both dense and TSVECTOR matching (AWS) (DataCamp) on queries like "multi-hit Fighting moves." The corpus structure (per-move, per-ability, per-Pokémon, per-team pages) is exactly the regime where Anthropic's technique shines.

Concrete implementation. At ingestion time, for each chunk call Claude Haiku with the *full parent doc* in a `{ cache_control: { type: "ephemeral" } }` block, followed by the chunk and the standard instruction: *"Please give a short succinct context to situate this chunk within the overall document for the purposes of improving search retrieval of the chunk. Answer only with the succinct context and nothing else."* (CO/AI) (Medium) Prepend the returned 50-100 tokens to the chunk and store both in a new `context` column for auditability. Re-embed, re-index FTS. Run once per document; incremental reruns only touch affected docs.

Economics: with Haiku and prompt caching (90% read discount), Anthropic reports about **\$1.02 per 1M document tokens**. (Claude +2) For a Pokémon corpus of roughly 20K chunks × 1K token docs, budget **\$20-40 one-off**. Files to touch: `lib/chunker.ts` (add `contextualize(chunk, doc)` step before embed), add `ctx` column to the chunks table, re-ingest.

Priority 2: Add a cross-encoder or ColBERT reranker

The top-40 results from the hybrid RRF RPC are **not ordered well enough for generation** — RRF is a fusion heuristic, not a relevance model. A cross-encoder reranker over the top-40 lifts nDCG@10 by +5-15 absolute points in BEIR-class evaluations, which is more than any other single change.

Options, ranked for this project:

1. **Jina Reranker v2 base multilingual** via `(POST https://api.jina.ai/v1/rerank)`. (OpenRouter) Cheapest per call (~\$0.02 / 1M tokens), (Getmaxim) 15× throughput of BGE-reranker-v2-m3, (AWS Marketplace) 100+ languages, (Jina AI) 1024-token window. (Jina AI) Example uplift: snowflake-arctic-embed-l baseline 0.6100 → **0.6481 nDCG@10** with jina v2 on top. (arXiv) Perfect Vercel fit — one `fetch()` call.
2. **Cohere Rerank 3.5** — \$2 / 1K searches (MetaCTO) (1 query + up to 100 docs). ~400-600 ms latency for ~100 docs. (Towards Data Science) (CustomGPT) Simplest TS SDK integration via `cohere-ai` v7+.
3. **answerai-colbert-small-v1** (Jina AI) (33M params, MiniLM-sized, 53.79% avg nDCG@10 on BEIR (PromptLayer) — beats ColBERTv2 at 110M) (Answer +3) run on a Python sidecar

(Modal/Replicate) or via Jina's hosted `jina-colbert-v2` API. Best raw quality on entity-rich out-of-domain queries.

4. `cross-encoder/ms-marco-MiniLM-L-6-v2` (Medium) via `@xenova/transformers` ONNX directly on Vercel — zero-cost, zero-dependency, but weak by 2026 standards (often *regresses* on strong first stages). Use only as a free sanity-check baseline.

Integration pattern (in `lib/rag.ts`, after the existing hybrid RPC):

```
ts

const { data: candidates } = await supabase.rpc('pc_hybrid_search', { p_fetch_k: 40,
const reranked = await rerank(query, candidates.map(c => c.content)); // HTTP call
const top8 = reranked.results.slice(0, 8).map(r => candidates[r.index]);
```

Keep the existing 8-boost additive re-ranker as a *post-rerank metadata bias layer* — they are solving different problems (cross-encoder = semantic relevance; additive boosts = metadata-aware priors like "usage intent + matching Pokémon"). Recalibrate the boost scale to the reranker's output range, which is typically [0, 1] not [0.02, 0.035].

Cache rerank results. Key: `sha256(normalize(query) || "||" || sorted(candidate_ids).join(","))`. Value: ordered `[candidate_id, score]` list. TTL tied to corpus version. On a slow-changing Pokémon corpus, cache hit rates of 40–70% are realistic.

Priority 3: Query transformations, routed by intent

Don't apply query transforms to every query — they have cost and they *hurt* on entity-heavy lookups.

- **HyDE** (arXiv 2212.10496): for stylistic-gap queries only ("best sweeper for rain teams"). Never for entity lookups ("moves learned by Garchomp at level 48") — the hallucinated passage invents wrong numbers and pollutes the dense query vector. Gate with intent classifier: HyDE only for `counter / team / strategy` intents, never for `stat / move / item`.
- **Step-back prompting** (arXiv 2310.06117): generate a more abstract version of the query, retrieve with both. (arXiv) (Learn Prompting) Reported gains are dramatic on multi-hop: TimeQA 42.6% → 68.7% (arXiv) with step-back + RAG. (arXiv) Best fit here for rules/mechanics questions ("Does Intimidate work on Substitute?" → step-back "How do abilities interact with Substitute in Gen 9?").
- **Multi-query / RAG-Fusion**: generate 3–5 paraphrased queries, retrieve each, union via RRF. Already compatible with the existing RPC — `Promise.all(queries.map(q =>`

`supabase.rpc('pc_hybrid_search', ...)))` and fuse. Strong for ambiguous queries ("best water starter"). +2-6% recall typical.

- **Query decomposition** for compound queries: "which Fire-type has higher SpA than Charizard and learns Solar Beam" → decompose into sub-questions. Most queries here are single-hop; reserve for detected multi-predicate intent.
- **Conversational query rewriting**: essential if the system enters chat mode. "What about its hidden ability?" needs rewrite to include the prior Pokémon. +10-30% MRR on conversational datasets (MaFeRw arXiv 2408.17072).

Hybrid tuning — keep k=60, but understand where BM25 wins

RRF k=60 is well-calibrated and rarely worth tuning. Much more important: ensure the **TSVECTOR side is optimal for entity recall**, because sparse retrieval is *the* documented strength for rare named-entity queries, which is exactly the Pokémon / move / item regime.

Concrete changes to the FTS column:

```
sql

setweight(to_tsvector('simple', name || ' ' || aliases), 'A') ||
setweight(to_tsvector('english', description), 'B') ||
setweight(to_tsvector('english', lore), 'D')
```

Use `'simple'` (no stemming) on name fields so "Garchomp" and "Tera Blast" are not butchered into "garchomp" → "garcom"-style stems. Reserve `'english'` for prose.

ParadeDB's `pg_search` (true BM25 on Postgres via Tantivy, 20× faster than tsquery+ts_rank at 1M rows) is *not* currently available on managed Supabase (#18061). For a Pokémon corpus under 500K chunks, `ts_rank_cd` with careful `setweight` is entirely sufficient. Revisit pg_search only if text search becomes a measured bottleneck.

Routing and fallback — upgrade the intent classifier

The rule-based classifier is correct for the well-enumerated head intents. For the long tail, port **Aurelio Labs' semantic-router** pattern into TypeScript: store ~5-10 labeled "utterances" per route in a small pgvector table (`routes(route_name, utterance, embedding)`), embed the user query, do a single k-NN lookup, apply a similarity threshold. Latency drops from ~500 ms (LLM router) to 10-100 ms (pure vector lookup). Only fall back to an LLM self-query classifier when the top-1 utterance similarity is below τ .

Confidence-driven fallback chain:

1. Rules match → structured SQL → return.

2. Semantic router match → dispatch to `(hybrid / sql / graph / web)`.
3. Run hybrid; compute confidence from `(top1_rrf_score)` (normalized) and gap between top-1 and top-5.
4. If top-1 below τ_1 or gap below τ_2 → trigger `(/research)` web fallback.
5. Still empty → refuse with "I don't know about X."

Calibrate τ_1, τ_2 empirically on a labeled held-out set of answerable vs. unanswerable queries, picking the PR knee.

Hierarchical / small-to-large

The Pokémon corpus has an obvious hierarchy: franchise → generation → Pokémon entry → move/ability section → competitive analysis. Implement `(parent_id uuid)` on the chunks table and a sibling `(documents)` table. Index at the child-chunk level for retrieval precision; return the parent chunk (full Pokémon entry or full team markdown) for generation context. One extra SQL join, no external docstore.

For long team writeups, use sentence-window retrieval: store `(doc_id, seq_no)`, and after ANN do `(WHERE doc_id = $1 AND seq_no BETWEEN $2-k AND $2+k)`.

RAPTOR (arXiv 2401.18059) reports +20 pts on QuALITY but requires LLM summarization per cluster per level — overkill here. A single handcrafted summary chunk per Pokémon and per team archetype gives 80% of the benefit for 1% of the indexing cost.

GraphRAG — use the existing schema, skip the framework

Microsoft GraphRAG (arXiv 2404.16130) and LightRAG (arXiv 2410.05779) shine on unstructured corpora where entity and relation extraction must be done by an LLM — which is the expensive part. **This project already has the graph in relational form:** Pokémon, moves, items, types, teams, team members, and — critically — **matchup matrices in CSV**. The matchup CSV is literally an adjacency list for a weighted typed directed graph.

Rather than running GraphRAG indexing (which would cost \$30–80+ in LLM calls for a corpus this size and re-run on every update), **promote the existing schema to a first-class graph** and traverse it with recursive CTEs in Postgres:

```
pokemon(id, name, ...)
moves(id, name, type, category, bp, ...)
pokemon_moves(pokemon_id, move_id)
type_matchup(attacker_type, defender_type, multiplier)
teams(id, archetype, regulation, ...)
team_members(team_id, pokemon_id, slot, item, tera, moves, ev_spread)
chunk_entities(chunk_id, entity_type, entity_id)  -- links free-text chunks to
IDs
```

Multi-hop queries like "counters to Dragapult" become graph walks: find P such that some move learned by P is super-effective against Dragapult's types AND Dragapult's moves are resisted/neutral against P. 1-3 hop traversals in `WITH RECURSIVE` run in < 10 ms on a well-indexed schema.

Use an LLM **only** to populate the `chunk_entities` link table at ingestion — given a chunk, extract `{entity_type, entity_id}` pairs against the canonical Pokémon/move dictionaries. That's the one place GraphRAG-style entity linking actually pays for itself here.

Late interaction / ColBERT

Direct ColBERT retrieval on pgvector is not feasible: no native MaxSim operator in vanilla pgvector, ~100× more stored vectors per chunk, no Node inference path. The only reasonable route is as a **reranker over top-50** via Jina's hosted `jina-colbert-v2` API (multilingual, Matryoshka 64-dim heads, one fetch call) or a Python sidecar running `answerai-colbert-small-v1` (33M, CPU-viable, BEIR 53.79). This is subsumed by the "Priority 2: reranker" choice above; pick ColBERT-style over classic cross-encoder if entity-rich out-of-domain queries dominate.

If you ever need native MaxSim in Postgres, **VectorChord** (pgvecto.rs successor) now ships a decomposed MaxSim implementation in SQL with a documented HNSW → ColBERT rerank flow, but migrating off Supabase-managed Postgres is a big operational change.

Lost-in-the-middle and context size

Liu et al. (arXiv 2307.03172) showed a U-shaped accuracy curve: performance is highest when the relevant passage sits at the start or end of the prompt and degrades in the middle, even on long-context models. Mitigations for this project:

- **Rerank so best chunks land at the edges** of the assembled context.
- **Keep context short** — 5-10 chunks, not 50.
- **Repeat the query after the context** ("query-aware contextualization").

Debugging retrieval failures

- *Recall failure* (gold chunk not in top-k): expand k to 200. If still absent, the embedder is off-domain or chunking destroyed the signal. Fix by smaller chunks, better encoder, entity-aware hybrid.
 - *Precision failure* (gold retrieved but buried): add or tune a reranker.
 - Visualize the embedding cloud with UMAP (n_neighbors=15) colored by data_category. Clumps = redundancy; stripes = leakage; singletons = orphans. Compare query embeddings vs corpus cloud to detect distribution shift.
-

4. Embedding and chunking upgrades

The baseline is behind SOTA

MiniLM-L6-v2 scores ~**56 MTEB avg** / ~**42 retrieval**, roughly 15 points behind modern retrieval SOTA. Trained with seq_len=128, truncates aggressively above 256 wordpieces. On short factoid Pokémon queries this is not catastrophic but it leaves ~10-20% recall on the table.

The recommended upgrade path

Non-negotiable first step: migrate the pgvector column from `vector(384)` to `halfvec(384)`. Added in pgvector 0.7.0, it is 2× smaller, builds HNSW ~2× faster, and costs < 1% recall on normalized cosine vectors (AWS Aurora and Neon benchmarks). Zero schema-level downstream changes.

Model upgrade — primary recommendation: EmbeddingGemma-300m with MRL truncation to 384 dims.

- 308M params, ~300 MB fp32, ~150 MB int8 ONNX — fits Vercel's 250 MB unzipped budget at q8.
- Native Matryoshka at {128, 256, 512, 768}. Truncate to 384 with L2 renormalization; quality loss at 768→384 is typically < 2% for MRL-native models.
- 100+ languages — handles Italian natively, retires the 2,383-entry dictionary.
- 2048-token context, strong retrieval.
- Official transformers.js / ONNX support via Xenova's builds.
- HF Inference API supported for the query-side path (which is already how queries are embedded on Vercel because `onnxruntime-node` bundles flaky).
- License: Gemma Terms of Service — review, usually acceptable.

The project previously rejected `embeddinggemma-300m-ONNX` at 768-dim q8 as too heavy; the new path is different — q8 ONNX at **384 MRL-truncated** is materially smaller, and the main motivation (Italian coverage + retrieval quality) has become clearer.

Alternative A — fastest 1-day upgrade with no schema change: `bge-small-en-v1.5` (33M, 384-dim, 512 ctx, MTEB 62.17, retrieval 51.68, Apache-2.0). Drop-in replacement for MiniLM-L6-v2, +10 retrieval points, ONNX-ready, zero operational change. Does *not* help with Italian. Requires the prefix `"Represent this sentence for searching relevant passages: "` on queries only; bake it into the query encoder and nowhere else.

Alternative B — purely OSS, stronger multilingual: `snowflake-arctic-embed-m-v2.0` (~305M, 768-dim native → MRL-trained for 256, Apache-2.0, 100+ languages incl. Italian, 8192 ctx). Schema migrates to `halfvec(256)`. Weaker transformers.js support (`trust_remote_code` path breaks in some JS runtimes); test carefully.

Alternative C — highest quality, API-only: `Qwen3-Embedding-0.6B` via HF Inference API only. SOTA multilingual, Apache-2.0, Matryoshka to 384. Removes local transformers.js path entirely.

Rejected: BGE-M3 (2.3 GB, dense output forces 1024-dim schema migration without using its hybrid-sparse value prop), jina-embeddings-v3 (CC-BY-NC-4.0 blocks commercial use), Stella/Qwen3-4B+/OpenAI-3-large (too big or no HF Inference API fit).

Migration order.

1. **Day 1:** `vector(384)` → `halfvec(384)`. Free 2× storage + faster HNSW build. No other changes.
2. **Week 1:** Bench `bge-small-en-v1.5` vs MiniLM on a held-out 50-query Pokémon set. If +10% nDCG@10, ship same-day as drop-in.
3. **Month 1:** Pilot EmbeddingGemma-300m q8 ONNX in transformers.js locally, measure Lambda cold-start and unzipped size. If under budget, dual-write to a shadow column, run side-by-side eval, cut over.
4. **Month 2:** After validating multilingual retrieval holds, **retire the 2,383-entry Italian dictionary**.

Matryoshka — the reason schema stability is cheap

MRL (Kusupati et al., NeurIPS 2022, arXiv 2205.13147) trains the first k dimensions to be a valid embedding themselves. Truncating a 768-dim MRL-native output to 384 and L2-renormalizing is nearly lossless (typically < 2% nDCG). This is the mechanism that lets the project keep its existing `halfvec(384)` HNSW index while upgrading models. **Always renormalize after truncation**, and ensure ingestion and query paths apply the *exact same* truncation + renorm sequence.

Caveat: truncating a **non-MRL** model (classic MiniLM, older BGE) loses 10-20%. MRL only helps on MRL-native models.

Instruction-tuned models — the query prefix is non-optional

Most modern encoders (E5, BGE, Arctic, GTE-Qwen, Qwen3, Jina-v3, EmbeddingGemma) require a task-specific prefix on queries. Skipping it costs 3-8 nDCG points. The prefix differs per model — bake it into the query-side encoder wrapper and document it in the ingestion pipeline so future changes don't silently drop it. MiniLM was *not* trained with prefixes, so dropping one now is fine; dropping one after upgrade is a silent bug.

Quantization on Supabase

Level	Size (768-dim)	Typical recall loss	pgvector type
fp32	3072 B	baseline	<code>vector(n)</code>
fp16	1536 B	< 1%	<code>halfvec(n)</code> 
int8	768 B	1-3%	<code>smallint[]</code> or <code>sparsevec</code>
binary	96 B	10-30% standalone; 2-5% with rerank	<code>bit(n)</code> with <code>bit_hamming_ops</code>

Move to `halfvec` immediately. Binary only becomes compelling at 1024+ dims with a rerank stage — not currently the bottleneck.

Chunking strategies — what to change in `lib/chunker.ts`

Research consensus (LlamaIndex, Chroma Research, NVIDIA 2024 7-strategy study, Firecrawl 2026 guide):

- **Factoid / short-answer queries: 256-512 tokens optimal.**
- **Analytical queries: 1024+ optimal.**
- **Chroma Research:** LLM/ClusterSemanticChunker 0.913-0.919 recall; RecursiveCharacterTextSplitter 85-89% at ~400 tokens. Max cross-method variance ~9%.
- **Overlap:** 10-20% of chunk size is the sweet spot; hurts > 25-30%.

Concrete changes for this project:

1. **Team writeups → one chunk per team (all 6 slots concatenated)**, with prepended metadata `[archetype | regulation | top_placement]`. Embedding each slot independently loses the team-level context that dominates `/team` queries ("teams using

Urshifu + Amoonguss"). If a team exceeds ~2K tokens with full EV/IV/move detail, use **small-to-large**: index a summary chunk, retrieve the full team markdown for generation.

2. **Pokémon stats / moves / abilities CSVs — keep row-per-chunk, add richer metadata prefix.** Current pattern is correct. Prepend `"[pokemon_name: Iron Valiant] [tier: VGC Reg G] [types: Fairy/Fighting]"` and every attribute you have; this is near-free and helps both lexical and semantic match on attribute queries. Store the same fields as structured Supabase columns so they can also be used as hybrid filters.
3. **Markdown meta/strategy articles — switch from "3-line trailing overlap" to `MarkdownHeaderTextSplitter`** that preserves the full H1 → H2 → H3 path as metadata on each chunk. Target ~400-512 tokens per chunk with 10-15% overlap. Fall back to recursive character splitting only inside sections that exceed the target.
4. **Type-matchup tables — render as NL prose at ingestion**, one chunk per attacking type: *"Fairy resists Fighting, is weak to Steel and Poison, immune to Dragon..."*
5. **Always use the `'simple'` analyzer for name fields in TSVECTOR** so Pokémon/move names aren't stemmed into nonsense.

Late chunking — the right next embed-side upgrade after EmbeddingGemma

Günther et al. (Jina AI + Weaviate, arXiv 2409.04701) proposed embedding the *full document* first through a long-context encoder, then mean-pooling per chunk-span at the token-embedding level. Each chunk's embedding is conditioned on cross-chunk context (headings, anaphora). Reported gains: +4-9% nDCG on BEIR, up to +23% on multi-hop/anaphoric queries. Works with any mean-pooled long-context model (EmbeddingGemma 2k, BGE-M3, Arctic v2).

Apply late chunking to team markdowns specifically. Queries like "this team's wallbreaker" → Iron Valiant only resolve correctly if the wallbreaker embedding knows it is part of a specific team. Late chunking solves this at no extra indexing cost beyond running the encoder on the full doc. Strict upgrade for anaphoric queries.

Contextual chunk enrichment — the cheap version

If Anthropic's Contextual Retrieval is too heavy for some slice of the corpus, use the two near-free enrichments:

- **Metadata prepending.** Deterministic, free, and best-ROI: embed `"[pokemon_name: X] [tier: Y] [types: Z] <chunk text>"`.
- **Hypothetical questions.** Generate 3-5 questions each chunk could answer, embed them alongside or instead of the chunk. Cheaper than Anthropic's method and ~half the gain.

Kill the Italian translation layer — but stage it

Short term (next 30 days): Keep the dictionary, but move it from *chunk-time mutation* to *query-time expansion*. Stored embeddings stay consistent as the encoder swaps; Italian queries get enriched with English translations before embedding. This decouples the dictionary's lifetime from the index's lifetime.

Medium term (post-EmbeddingGemma rollout): Retire the dictionary. Validate with a 50-query bilingual eval set (same factual question in both languages); if cross-lingual nDCG@10 is within 2% of monolingual, cut it. Do not invest in growing the dictionary — the embedding-model-as-translator path is strictly better at Pokémon-specific vocabulary because these models saw Pokémon text during pretraining while the dictionary only sees what was curated from Pikalytics.

5. Agentic RAG patterns

The agentic layer (`/lookup`, `/team`, `/calc`, `/research`), CLAUDE.md persona, loop detection, pokedex dedup, post-loop force-completion) already addresses the classic failure modes: retrieval loops, over-retrieval, citation-by-dedup. The highest-leverage additions, in priority order:

Priority 1 (cheapest + highest trust): a groundedness / citation-ID validation loop

Force the generator to emit answers in structured form:

```
{ "answer": "...",
  "claims": [
    { "text": "Garchomp learns Earthquake by TM", "chunk_ids":
["pdx_garchomp_moves_42"] },
    ...
  ]
}
```

Server-side, validate each `chunk_id` against the actual retrieval set. On any invalid citation, either auto-retry with a reminder prompt or downgrade confidence and flag. Optionally add a post-hoc Haiku pass: "For each claim, is it supported by the evidence? Return `{claim, supported: true|false}`". Flag answers with < 80% supported; trigger a Chain-of-Verification refinement.

Chain-of-Verification (CoVe, Dhuliawala et al., arXiv 2309.11495, ACL 2024) is the rigorous form: draft → plan verification questions per claim → answer each independently *without seeing the draft* → revise. Reported: Llama 65B list-QA precision 0.17 → 0.36, biography FactScore 55.9 →

71.4. CoV-RAG variant adds retrieval to the verification step, +3.7–4.0 points on NQ/WebQ/Mintaka/TriviaQA.

For this project, **start with forced-JSON citation output + post-hoc Haiku groundedness check**, then add full CoVe only on high-stakes modes (`/team evaluate`, `/team counter`). Latency +200 ms, cost +~5%, blocks most remaining hallucinations.

Priority 2: Plan-and-Execute for `/team` modes (LLMCompiler-style)

LLMCompiler (Kim et al., ICML 2024, arXiv 2312.04511) decomposes a query into a DAG of tool calls with `$variable` placeholders, executes independent nodes in parallel. Reports vs ReAct: **3.7× latency, 6.7× cost, +9% accuracy** on HotpotQA / Movie Rec / ParallelQA, and beats OpenAI native parallel function calling by 1.35×.

For `/team build "rain team around Pelipper"`, a ReAct loop serially runs: `lookup(Pelipper)` → `lookup(rain abusers)` → `lookup(Swift Swim mons)` → `lookup(counters to rain)`. An LLMCompiler plan does 2–4 in parallel. Claude 4+ does parallel tool use natively — you get most of the benefit by (a) adding an explicit Planner phase (one call to emit the plan as JSON) and (b) relying on Claude's built-in parallel `tool_use` for independent siblings. Direct user-visible latency improvement on the heaviest operation in the product.

Priority 3: Self-RAG-style "Retrieve?" gate (prompted, no fine-tune)

Self-RAG (Asai et al., ICLR 2024 Oral, arXiv 2310.11511) trains a model to emit reflection tokens (`Retrieve`, `IsRel`, `IsSup`, `IsUse`). The project can't fine-tune, but it can **emulate the gate** via structured JSON on Claude:

```
{ "retrieve": true|false|"continue",  
  "isRel": "relevant"|"irrelevant"|"partial",  
  "isSup": "full"|"partial"|"none",  
  "isUse": 1..5,  
  "rationale": "..." }
```

If `retrieve=false` and the rule-based intent classifier agrees, skip the pgvector call (big latency + cost win on chat-style queries). If `isSup != "full"`, trigger a reformulation and re-retrieve. Cost: +1 short structured LLM call (~200 tokens) with prompt caching. Targets `phantom_pokemon` and `stat_accuracy` regressions specifically.

Priority 4: Subagent decomposition via the Claude Agent SDK

Migrate `/team build`, `/team evaluate`, `/team counter` into distinct subagents with:

- Focused system prompts per mode.

- Restricted tool allowlists (`evaluate` has `/calc` + `pgvector`; `build` has `pgvector` + `/research`; `counter` has `pgvector` + `matchup CSV graph`).
- Optionally different models (Haiku for `build` exploration, Sonnet for `evaluate`).

Fresh context per subagent, only the final message returns to the parent. Opus tends to over-spawn subagents; give explicit "when NOT to delegate" guidance. Consider migrating the CLAUDE.md persona to a `.claude/skills/pokemon/SKILL.md` for progressive disclosure — only the relevant references load when a matching query arrives.

Priority 5: Prompt caching with the 4-breakpoint structure

Anthropic's prompt caching: cache reads cost **10%** of base input, cache writes **125%** (5-min TTL) or **200%** (1-hour TTL). Up to 4 `cache_control: { type: "ephemeral" }` markers per request; minimum cacheable block 1,024 tokens (Sonnet/Opus) / 2,048 tokens (Haiku). Recommended layout:

```
[ tools ]                ← breakpoint 1 (rarely changes)
[ system + CLAUDE.md persona ] ← breakpoint 2 (near-static)
[ retrieved context block ] ← breakpoint 3 (per-query)
[ conversation history + query ] ← breakpoint 4 (incremental)
```

Economics on Sonnet-class pricing: cold call 9K tokens at $1.25 \times = \$0.034$; warm same-session different-query same-retrieval = cached read $8.5K \times 10\% = \$0.003$ — **~90% savings**, matching Anthropic's published claim. Even when retrieval changes but persona/tools don't, ~50% savings persist.

Watch-outs: any change to `tool_choice`, any image, any change to thinking params, and any token change *before* a breakpoint invalidates that breakpoint and everything after. Keep `tool_choice` stable per session; version the persona (`<!-- v3 -->`) so you know exactly when BP2 invalidated.

This matters more once the Claude fallback path becomes primary; today the default is gemma-4-26b via OpenRouter. Implement now so the Claude path is economical when `/team evaluate` escalates.

Priority 6: Corrective RAG (CRAG) via Haiku retrieval grader

CRAG (Yan et al., arXiv 2401.15884) replaces T5-Large with a Haiku call that grades each retrieved chunk on {correct, ambiguous, incorrect}. Correct → decompose-recompose (strip-level filtering to remove noise within chunks). Incorrect → discard and fall back to web search (your existing `/research` command). Ambiguous → union. Haiku at ~\$1/M in is cheap enough to grade 5-10 chunks per turn in a single batched call (~\$0.0005, 300 ms).

For this project, the "web fallback" maps cleanly onto `/research`; CRAG formalizes *when* to invoke it. Targets `tournament_retrieval`, `meta_core_attribution`, `usage_teammates` when retrieved chunks are mixed relevance.

Priority 7: IRCOT for multi-hop `/team counter` queries

Interleaving Retrieval with Chain-of-Thought (Trivedi et al., arXiv 2212.10509): each new CoT sentence becomes the next retrieval query. Reports +21 points recall, +15 points QA on HotpotQA / 2WikiMultihopQA / MuSiQue / IIRC. Fits naturally on `/team counter "stall teams in Gen 9 OU"`: identify stall archetypes → identify members → identify shared weaknesses → recommend breakers. Only implement after measuring whether `usage_teammates` and `meta_core_attribution` failures are actually 2-hop failures.

Tool-use with Claude: schema and choice discipline

From Anthropic's "Writing tools for agents" and the current Claude Agent SDK docs:

- **Tool choice.** `auto` for `/lookup`, `/research`; `any` to force *some* tool on ambiguous queries; `tool: { name }` to force pgvector first in `/team` modes; `none` for chit-chat. **`any` and `tool` are incompatible with extended/adaptive thinking** — use `auto` or `none` with thinking.
- **Schema best practices.** Unambiguous parameter names (`user_id` beats `user`). Describe as if to a new hire, including domain jargon. Use `strict: true` to eliminate invalid calls. Cap response size (Claude Code default = 25K tokens); paginate / filter / truncate with sensible defaults. Make error and truncation messages *steer* ("try more specific queries") rather than return opaque error codes.
- **Parallel tool use.** Good by default on Claude 4+. Never insert text before `tool_result` blocks in message history — it teaches Claude to serialize.
- **Extended / adaptive thinking + tools.** Pass thinking blocks *unmodified* back on subsequent calls. Thinking blocks don't support forced `tool_choice`. Any change to `tool_choice`, the presence or absence of images, or thinking params invalidates cache.
- **The `think` tool.** A no-op that appends a thought to the trajectory — useful specifically when the model should reflect on tool output mid-loop. Anthropic now recommends extended/adaptive thinking over the `think` tool in most cases, except when reflection on a just-returned tool result is needed.

Failure modes, symptom-to-fix map

Failure	Symptom	Mitigation
Retrieval loop	Same RPC call 3+ times	Already detected; force completion (already done).
Context stuffing	Degraded recall at > ~30 chunks	CRAG decompose-recompose; cap top-K at 8 after rerank.
Citation hallucination	Chunk ID that was never retrieved	Forced JSON with <code>chunk_ids</code> , validated server-side against retrieval set.
Over-retrieval	pgvector called for chit-chat	Self-RAG-style <code>Retrieve?</code> gate + existing rule classifier.
Under-retrieval	Parametric answer on time-sensitive query	Intent classifier routes usage/tournament/meta to forced retrieval via <code>tool_choice: "any"</code> .
Tool schema confusion	Wrong tool, malformed args	<code>strict: true</code> ; examples in tool description.
Cache invalidation	<code>tool_choice</code> flipping kills prefix cache	Keep <code>tool_choice</code> stable within a session; stable JSON key ordering in tool-use blocks.
Sycophantic self-critique	Model says "looks good" to its own output	CRITIC (arXiv 2305.11738): external tool verification beats pure Self-Refine.

Specifically: **avoid pure Self-Refine**. "Large reasoning models cannot reliably self-correct reasoning" (Huang et al., arXiv 2310.01798) is the counterpoint to the Reflexion/Self-Refine family. Favor **grounded verification** — CRITIC for external tool checks, CoVe for independent verification questions, forced citations for structural grounding.

Extending the 13-test agentic harness

Add, in priority order:

1. **Trajectory metrics** (`tool_call_count`, `unique_chunk_ids_used`, `retrieval_precision@k`) to catch over-retrieval and loops that the final-answer check misses.
2. **Groundedness score** per test (fraction of claims with valid chunk citations).
3. **`pass^3` consistency** (τ -bench-inspired, arXiv 2406.12045) — run each test three times at temperature > 0 and measure agreement. Most important agent-reliability metric in 2025.
4. **Cost/latency budgets** per test — regress if a new prompt bloats tokens or wall time.

5. **Adversarial cases** — prompt-injection-in-chunk, similar-name phantom Pokémon, typos.
 6. **Cache-hit regression** — assert `usage.cache_read_input_tokens > 0` on expected-cached calls.
-

6. Benchmarks the playbook refers to

- **BEIR** (Thakur et al., 2021, arXiv 2104.08663) — 18-dataset heterogeneous retrieval benchmark. Current baseline anchor for nDCG@10 uplift claims.
 - **MTEB / MMTEB** (Muennighoff et al., arXiv 2210.07316; MMTEB arXiv 2502.13595) — de-facto embedding benchmark. Use retrieval subset, not overall average.
 - **MultiHop-RAG** (Tang & Yang, COLM 2024, arXiv 2401.15391) — multi-hop queries over news, four types (inference, comparison, temporal, null). Standard RAG+GPT-4 only ~50% accuracy; retrievers miss 20-40% of sub-claim evidence.
 - **CRAG benchmark** (Meta, 2024) — real-world QA with noisy retrieval; drives the "retrieval grader" pattern.
 - **RGB** (Chen et al., arXiv 2309.01431) — noise robustness, negative rejection, info integration, counterfactual robustness. Exactly the axes current evals miss.
 - **τ -bench / τ^2 -bench / τ^3 -bench** (Sierra, arXiv 2406.12045 and subsequent) — agent reliability via `pass^k`; τ^3 adds knowledge retrieval and voice.
 - **AgentBench** (Liu et al., arXiv 2308.03688) — 8-environment agent benchmark; establishes open-source-vs-API gap.
-

7. Prioritized action plan — what to do next for THIS project

Ranked by impact ÷ effort, with files and effort estimate:

P0 — do this week

1. **Replace saturated MRR eval.** Switch scoring to nDCG@10 + Recall@10 + Context Precision@10 with graded relevance {0,1,2,3}. Add hard negatives to the 25-case set via the two-condition mining rule (arXiv 2505.18366). Effort: 1 day. Files: `scripts/eval.ts`. Impact: Immediately breaks the saturation.
2. **Migrate pgvector column to `halfvec(384)`.** Free 2× storage, 2× HNSW build speed, < 1% recall cost. Effort: half-day including reindex. Files: migration SQL, `lib/chunker.ts` write path.

3. **Add a cross-encoder reranker.** Jina Reranker v2 base multilingual via hosted API. Fetch top-40 from existing RPC → rerank → top-8. Keep existing 8-boost additive layer as post-rerank metadata bias, recalibrated to the reranker's [0,1] output range. Effort: 1 day. Files: `lib/rag.ts`. Impact: +5-10 nDCG@10.
4. **Wire Langfuse TS SDK + user feedback loop.** Instrument chat UI with `LangfuseWeb.score({ name: "user_thumbs" })`; propagate `traceId` through the RAG pipeline; always `shutdownAsync()` in Vercel handlers. Effort: 1-2 days. Impact: unlocks every subsequent metric.

P1 — do this month

5. **Contextual Retrieval at ingestion.** Haiku + prompt caching to generate 50-100 token chunk context; prepend before embed and FTS. Expected -35-49% retrieval failure. Effort: 2-3 days + \$20-40 one-off re-ingestion. Files: `lib/chunker.ts`, schema (`context` column), `scripts/reindex.ts`.
6. **Groundedness / citation-ID validation loop.** Force JSON output with `claims[].chunk_ids`; validate server-side; auto-retry on invalid. Post-hoc Haiku groundedness check for high-stakes modes. Effort: 2 days. Files: answer-formatter prompt, `lib/rag.ts` post-processing.
7. **Generate a ~200-case stratified testset via Giskard RAGET** (Python offline job), import into Langfuse datasets. Effort: 2 days. Impact: per-intent, per-difficulty visibility.
8. **Promote metadata prepending + MarkdownHeaderTextSplitter.** Every chunk embeds with `[pokemon_name] [tier] [types] [generation]` prefix; markdown chunking preserves full H1→H2→H3 path as metadata. Effort: 1-2 days. Files: `lib/chunker.ts`.
9. **Team writeups become one chunk per team** (all 6 slots concatenated) with metadata prefix. Effort: 1 day. Files: `lib/chunker.ts`.

P2 — next 1-2 months

10. **Embedding model upgrade to EmbeddingGemma-300m q8 ONNX, MRL-truncated to 384.** Dual-write, A/B eval, cut over. Effort: 1 week. Impact: +10-15% retrieval + retires Italian dictionary.
11. **Retire the Italian dictionary** after validating cross-lingual nDCG@10 parity (within 2% on a 50-query bilingual set). Effort: 2 days.
12. **Plan-and-Execute for `/team` modes** — explicit Planner call emitting a DAG; parallel `tool_use`. Effort: 2-3 days. Impact: 2-3× latency on `/team build|counter`.

13. **Promote the schema + matchup CSV to a first-class relational graph** (pokemon_moves, type_matchup, team_members, chunk_entities link table). Recursive CTE queries for "counters to X." Effort: 3-5 days. Impact: unlocks multi-hop queries at <10 ms, no LLM graph-indexing cost.
14. **Self-RAG-style `Retrieve?` gate** (prompted) for long-tail chit-chat detection. Effort: 1 day.
15. **Prompt caching with 4-breakpoint layout** on the Claude fallback path. Effort: 1 day. Impact: ~90% cost reduction on Claude.

P3 — opportunistic

16. **Subagent decomposition** (Claude Agent SDK) for `/team build|evaluate|counter`. Effort: 1 week.
17. **Late chunking** with EmbeddingGemma once deployed (chunk-after-embed at the token level). Effort: 2 days. Impact: +4-9% nDCG, more on anaphoric queries.
18. **Semantic-router pattern in pgvector** as a long-tail fallback to the rule classifier. Effort: 2 days.
19. **CRAG-style Haiku retrieval grader**. Effort: 2 days.
20. **IRCoT** for `/team counter` multi-hop queries — only after confirming 2-hop failures exist.
21. **Python RAGAS / RAGChecker offline job** in GitHub Actions for monthly claim-level diagnostics. Effort: 2-3 days.

What to deliberately skip

- **Microsoft GraphRAG or LightRAG** as full indexing pipelines — redundant with the existing relational schema; pay-for-nothing on \$30-80+ indexing cost per update.
 - **RAPTOR** — corpus is too small and already hierarchical.
 - **BGE-M3 / jina-v3** — BGE-M3 dense-only mode wastes its value prop; jina-v3 license (CC-BY-NC-4.0) is a blocker.
 - **ColBERT as primary retriever** — pgvector has no native MaxSim, ~100× more vectors. Use only as a reranker via Jina's hosted API if needed.
 - **ParadeDB `pg_search`** unless migrating off managed Supabase. `ts_rank_cd` + careful `setweight` is sufficient under 500K chunks.
 - **ARES** until the golden set exceeds 500 cases — its statistical-rigor value prop doesn't activate earlier.
 - **Fine-tuning the embedder** before exhausting the cheaper wins above.
-

8. Key references

Core papers. Self-RAG arXiv 2310.11511 · CRAG arXiv 2401.15884 · HyDE arXiv 2212.10496 · Step-back arXiv 2310.06117 · IRCot arXiv 2212.10509 · CoVe arXiv 2309.11495 · ReAct arXiv 2210.03629 · Reflexion arXiv 2303.11366 · Self-Refine arXiv 2303.17651 · CRITIC arXiv 2305.11738 · Plan-and-Solve arXiv 2305.04091 · LLMCompiler arXiv 2312.04511 · RAGAS arXiv 2309.15217 · RAGChecker arXiv 2408.08067 · ARES arXiv 2311.09476 · G-Eval arXiv 2303.16634 · FActScore arXiv 2305.14251 · RGB arXiv 2309.01431 · MultiHop-RAG arXiv 2401.15391 · RAPTOR arXiv 2401.18059 · ColBERT arXiv 2004.12832 · ColBERTv2 arXiv 2112.01488 · Jina-ColBERT-v2 arXiv 2408.16672 · RankGPT arXiv 2304.09542 · RankZephyr arXiv 2312.02724 · GraphRAG arXiv 2404.16130 · LightRAG arXiv 2410.05779 · Lost-in-the-middle arXiv 2307.03172 · Matryoshka arXiv 2205.13147 · Late Chunking arXiv 2409.04701 · jina-embeddings-v3 arXiv 2409.10173 · BGE-M3 arXiv 2402.03216 · Arctic Embed 2.0 arXiv 2412.04506 · Qwen3-Embedding arXiv 2506.05176 · EmbeddingGemma arXiv 2509.20354 · Dense X Retrieval (propositions) arXiv 2312.06648 · τ -bench arXiv 2406.12045 · AgentBench arXiv 2308.03688 · Position bias in LLM judges arXiv 2406.07791 · Hard negatives arXiv 2505.18366 · MMTEB arXiv 2502.13595.

Anthropic engineering. Contextual Retrieval (anthropic.com/news/contextual-retrieval) · Prompt caching (docs.anthropic.com) · Building effective agents · Writing tools for agents · Effective context engineering · The "think" tool · Building agents with the Claude Agent SDK · Code execution with MCP · Subagents / Skills in the Agent SDK.

Framework docs. RAGAS (docs.ragas.io) · RAGChecker (github.com/amazon-science/RAGChecker) · DeepEval (deepeval.com) · Langfuse TS SDK (langfuse.com/docs/observability/sdk/typescript) · Arize Phoenix TS (arize.com/docs/phoenix/sdk-api-reference/typescript) · Giskard RAGET · Aurelio Labs semantic-router (github.com/aurelio-labs/semantic-router) · RAGatouille (github.com/AnswerDotAI/RAGatouille) · LangChain ParentDocumentRetriever / SelfQueryRetriever · LlamaIndex HierarchicalNodeParser / SentenceWindowNodeParser · MTEB leaderboard (huggingface.co/spaces/mteb/leaderboard) · pgvector 0.7+ release notes (github.com/pgvector/pgvector) · VectorChord MaxSim (docs.vectorchord.ai) · Supabase pgvector docs · ParadeDB pg_search · Jina AI blog (late chunking, reranker v2, ColBERT v2) · Cohere Rerank 3.5 changelog · Voyage rerank-2 / 2.5 / context-3 blogs · Microsoft GraphRAG (microsoft.github.io/graphrag) · LightRAG (github.com/HKUDS/LightRAG).

9. Closing takeaways

The project is a mature, well-architected hybrid RAG system whose next order-of-magnitude improvements are not in the retrieval fusion layer but in **three adjacent layers**: the **ingestion layer** (Contextual Retrieval + richer metadata prepending + team-as-chunk), the **reranking layer** (adding a cross-encoder or ColBERT-style reranker that currently does not exist), and the

evaluation layer (escaping MRR 1.0 with graded relevance, stratified synthetic generation, and faithfulness scoring in Langfuse). Each of these unlocks the next: a real reranker only pays off when contextual embeddings give it better candidates to work with; a real reranker cannot be tuned without a real eval; and the eval cannot catch generation-side bugs without groundedness scoring and a groundedness-enforcing agentic loop.

The embedding upgrade (MiniLM-L6 → EmbeddingGemma-300m MRL-384) closes a ~15-point retrieval gap, retires the 2,383-entry Italian dictionary, and preserves the existing `halfvec(384)` HNSW schema thanks to Matryoshka truncation — a clean, low-risk migration.

On the agentic side, the existing loop-detection and dedup guardrails already handle the main failure classes identified in the 2024–2026 literature; the remaining wins are **grounded verification** (forced citations + CoVe-lite) and **parallelism** (Plan-and-Execute for `/team` modes). Prompt caching becomes a near-free 90% cost reduction the moment the Claude fallback path becomes primary.

Everything else — GraphRAG, RAPTOR, ARES, ColBERT as a primary retriever, fine-tuning — is more impressive than it is useful for this corpus at this scale. Skip them, and ship the P0/P1 list.